

Unity 2D 맵 시스템 UML & 디자인 패턴 기술 보고서

UML Diagrams, Structural Design Patterns & Architecture Blueprint

프로젝트명

unity_project_03team

분석 대상

Map Engine System

기술 표준

GoF & Unity Patterns / UML
2.5

코드 배제

Pure Architecture Spec (No C#
Code)

1. 개요 및 계층형 패키지 아키텍처 (Executive Summary & Package Architecture)

본 보고서는 **unity_project_03team** 프로젝트의 맵 엔진(Map Engine) 시스템을 **C#** 소스 코드를 완전히 배제하고, 객체 지향 공학 표준 **UML 2.5 다이어그램** 및 **GoF 디자인 패턴(Design Patterns)**만을 사용하여 분석 및 서술한 공식 기술 명세서입니다.

맵 시스템은 동적 무작위 생성, 메모리 파편화 방지, 물리 관성 동기화 및 2.5D 카메라 원근 연출을 달성하기 위해 역할별로 명확히 격리된 **5계층 패키지 아키텍처(Layered Package Architecture)**를 채택하고 있습니다.

UML PACKAGE & LAYERED ARCHITECTURE DIAGRAM

Presentation Layer

Cinemachine Confiner
Parallax Visuals

Event Bus Layer

MapEvent Bus
Spawn Triggers

Core Manager Layer

MapManager Singleton
ShuffleBag &
ObjectPool

Data / SO Layer

SOMapData Container
SOMapPalette &
MapBaker

Physics & Gimmick

MovingPlatform
SpikeTraps & JumpPads

2. 8대 핵심 디자인 패턴 명세 & 1:1 아키텍처 매핑 (Design Pattern Catalog)

맵 시스템의 확장성, 결합도 분리(Decoupling), 고성능 무가비지(Zero GC) 연산을 보장하기 위해 적용된 8가지 핵심 디자인 패턴 카탈로그입니다.

1. Singleton Pattern (싱글톤 패턴)

Creational Pattern

적용 클래스: MapManager

설계 의도: 맵 생성, 오브젝트 풀, 셔플백 및 진행 타이머 관제를 게임 전역에서 단 하나의 통제 인스턴스로 유지하고 인스펙터/씬 참조를 단일화합니다.

핵심 메커니즘: Awake() 생명주기 타이밍에 기존 인스턴스 중복을 검사(Self-Destroy)하여 유일성을 보장하고 static 접근 포인트를 제공합니다.

2. Object Pool Pattern (오브젝트 풀 패턴)

Creational / Behavioral Pattern

적용 클래스: MapManager (Internal Dictionary<GameObject>, Queue<GameObject>>)

설계 의도: 빈번한 맵 청크 동적 생성(Instantiate)과 파괴(Destroy)로 인한 메모리 파편화 및 런타임 가비지 컬렉션(GC) 렉을 완전히 억제합니다.

핵심 메커니즘: 씬 진입 시 맵 청크 프리팹을 비활성화 형태로 사전 생성(Pre-warming) 후 Queue 자료구조에 대여/반납 수명 주기를 관리합니다.

3. ShuffleBag / Random Bag Pattern (셔플백 패턴)

Behavioral Pattern

적용 클래스: MapManager (Internal List<GameObject> shuffleBag)

설계 의도: 난수 생성 기반 맵 동적배치 시 동일한 맵 청크가 연달아 출력되는 편향성을 방지하고 모든 맵이 균등한 확률로 출현하도록 보장합니다.

핵심 메커니즘: 등록된 프리팹 목록을 주머니(Bag)에 채운 뒤 무작위로 하나씩 뽑아 사용하며, 주머니가 완전히 비었을 때 재충전(Refill)하는 테트리스 무작위 방식을 적용합니다.

4. Observer / Event Bus Pattern (옵서버 / 이벤트 버스 패턴)

Behavioral Pattern

적용 클래스: MapEvent, MapChunkBoundaryTrigger, MapManager

설계 의도: 플레이어 이동 감지 컴포넌트와 맵 생성 매니저 간의 직접적인 의존성(Coupling)을 끊고 델리게이트 이벤트 기반 메시지 전달을 수행합니다.

핵심 메커니즘: MapEvent.onPlayerHitSpawnTrigger 이벤트를 발행(Publish)하고 MapManager가 이를 구독(Subscribe)하여 맵 생성 로직을 독립적으로 트리거합니다.

5. Flyweight / Data Container Pattern (플라이웨이트 패턴)

Structural Pattern

적용 클래스: SOMapData, SOMapPalette, SOMapPhaseSpawnSetting

설계 의도: 맵 청크의 무거운 메타데이터 및 스폰 정적 지점 정보를 런타임 게임오브젝트 인스턴스와 분리하여 메모리 공유 및 레벨 데이터 관리 효율화를 달성합니다.

핵심 메커니즘: 씬 메모리에 중복 데이터를 복제하지 않고 Unity ScriptableObject 에셋을 경량 공유 컨테이너로 참조합니다.

6. Double Buffering / Camera Blending Pattern (더블 버퍼링 패턴)

Structural / Rendering Pattern

적용 클래스: Cinemachine Confiner Blend System

설계 의도: 인접 맵 청크로 이동 시 카메라 Confiner 바운더리가 급작스럽게 교체되며 발생하는 화면 튕는 현상(Camera Snap)을 시각적으로 차단합니다.

핵심 메커니즘: 주 카메라와 서브 카메라 2대를 교대로 준비(Double Buffering)하고, Confiner 교체 시 Priority 수치 조절 및 0.5초 Blending을 통해 부드럽게 화면을 래핑합니다.

7. Builder / Extractor Pattern (빌더 / 세리얼라이저 패턴)

Creational / Behavioral Pattern

적용 클래스: MapBaker (Editor Utility)

설계 의도: 씬 하이라키에 시각적으로 배치된 레벨 요소(플랫폼, 가시, 스폰지점)를 자동 분석하여 S0MapData 에셋 데이터 구조체로 조립/직렬화합니다.

핵심 메커니즘: 하위 트랜스폼 탐색, 컴포넌트 데이터 추출 및 Snap 앵커 연산을 거쳐 복합 데이터 에셋을 단일 조립 절차로 생성합니다.

8. Strategy / Phase State Pattern (전략 패턴)

Behavioral Pattern

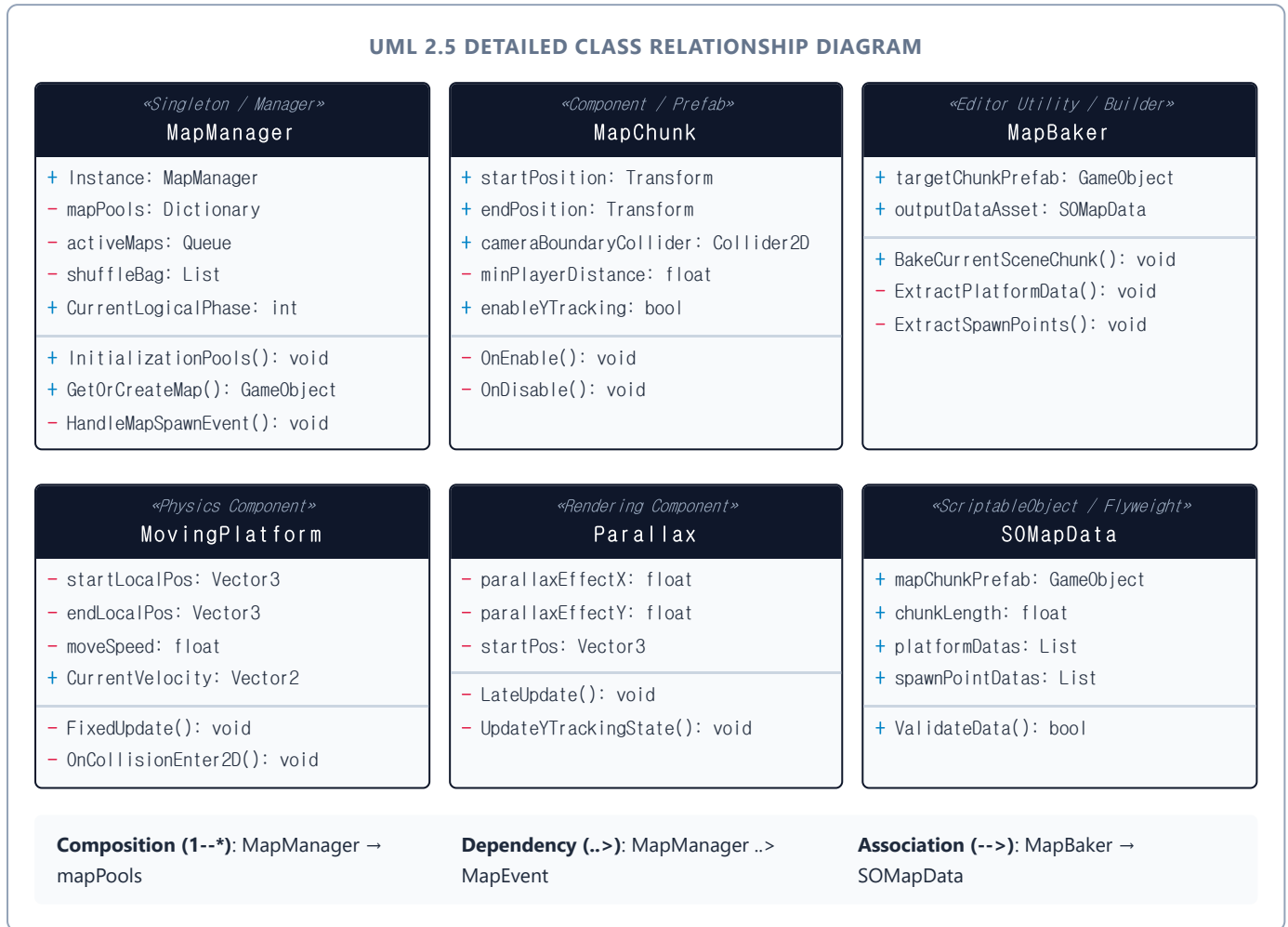
적용 클래스: MapManager.CurrentLogicalPhase 및 Phase Scaling Subsystems

설계 의도: 게임 진행 누적 시간에 따라 난이도, 트랩 주기, 몬스터 스폰율 및 카메라 Parallax Y축 제어 알고리즘을 런타임 동적으로 캡슐화 교체합니다.

핵심 메커니즘: 3단계 조건 매핑 함수를 통해 상위 논리 페이즈(Phase 0, 1, 2) 전략 객체를 스위칭하여 난이도 곡선을 적용합니다.

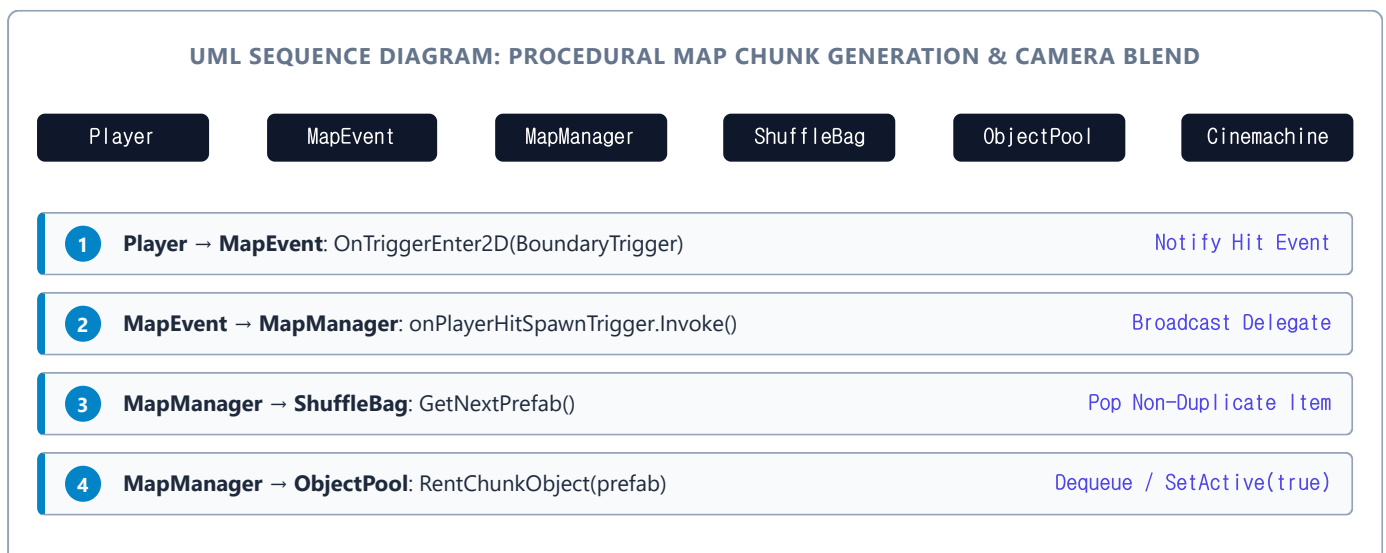
3. UML 정밀 클래스 다이어그램 (UML Class Diagram)

맵 시스템을 구성하는 핵심 C# 클래스 및 ScriptableObject 데이터 타입 간의 캡슐화 멤버 속성/메서드 서명 및 관계선(상속, 합성, 연관, 의존) 다이어그램입니다.



4. UML 동적 시퀀스 파이프라인 (UML Sequence Diagram)

플레이어가 스폰 트리거를 통과할 때 객체 간 발생하는 시점별 메시지 교환 파이프라인 시퀀스 다이어그램입니다.



5 **MapManager** → **MapChunk**: `SetPosition(lastEndAnchor.position)`

HalfGrid Snap Align

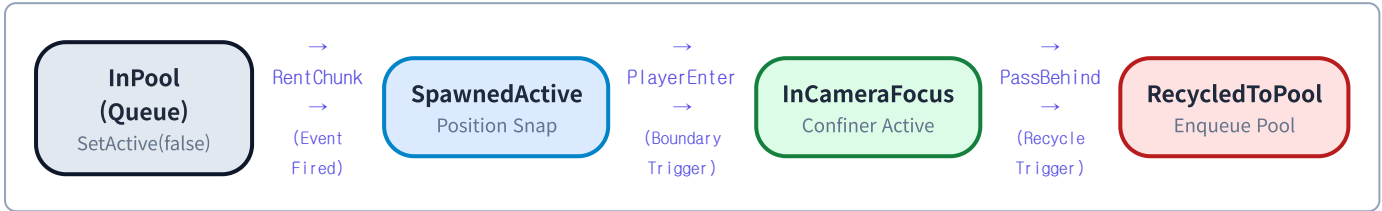
6 **MapManager** → **Cinemachine**: `UpdateConfinerBoundary(newCollider)`

Dual Camera 0.5s Blend

5. UML 상태 머신 & 맵 스케일링 메커니즘 (UML State Machine)

MapChunk 생명주기 객체의 상태 전환을 서술하는 UML State Machine Diagram 및 누적 시간 기반 CurrentLogicalPhase 전략 전환 구조입니다.

MapChunk 수명 주기 UML State Machine Diagram



Phase Strategy Execution Matrix

논리적 페이지 (State)	누적 시간 조건	적용 전략 (Strategy Policy)	UML 디자인 패턴 연동
Phase 0 State	0초 ~ 60초 미만	평지 위주, Y-Parallax 비활성화, 고빈도 상자 스폰	Strategy Pattern / Flyweight Palette 0
Phase 1 State	60초 ~ 120초 미만	이동 플랫폼 왕복, Y-Parallax 활성화, 측수 주기 단축	Strategy Pattern / MovingPlatform Physics
Phase 2 State	120초 이상	좁은 가시 발판, 고난도 정밀 점프, 측수 최고속도	Strategy Pattern / Double Confiner Blend

© 2026 unity_project_03team Development Team. All Rights Reserved. Verified Pure UML & Design Pattern Architecture Specification.